

Expresiones y Control de Flujo

Programación JAVA.

Variables y su ámbito

- Tipos de variables:
 - Según el tipo de datos
 - Tipos primitivos → Tipos de datos primitivos
 - Tipos de referencia → Referencian a objetos (punteros)
 - Según el entorno:
 - Variables declaradas dentro de un método de una clase
 - Variables locales
 - Variables declaradas fuera de un método, pero dentro de una clase
 - Variables de clase o atributos

Variables y su ámbito

- La variables locales son:
 - Variables definidas dentro de un método.
 - Se llaman variables locales, automáticas, temporales o de pila.
 - Variables creadas cuando se ejecutan los métodos y destruidas cuando termina el método.
 - Al ser variables locales a un método, se pueden utilizar múltiples variables que reciban el mismo nombre, puesto que su ámbito de existencia es definido
 - No se puede acceder a estas variables desde otros métodos
 - **Variables que deben ser inicializadas antes de ser usadas**
- También se consideran variables locales los parámetros de los métodos y los constructores, aunque éstas no tienen que ser inicializadas de forma explícita, puesto que ya lo son por el propio código
 - Se crea una variable de este tipo cuando se invoca un método de un objeto y existe mientras se ejecuta dicho método

Variables y su ámbito.

- Tipos de variables creadas fuera de un método, pero dentro de una clase:
 - Variables estáticas o variables de clase.
 - Se crean cuando se carga la clase y se mantienen mientras la clase está cargada en memoria.
 - Se indican con la palabra clave *static*
 - Se profundizará posteriormente en este concepto
 - Variables de instancia o atributos
 - Se crean cuando se instancia un objeto con la sentencia *new* y se mantienen mientras el objeto está referenciado.
 - Se denominan variables miembro o atributo, o variables de instancia
 - Son accesibles para todos los métodos de la clase
 - Se inicializan automáticamente cuando se crean
 - A *cero* los números, a *false* los boolean, a *\u0000* los char y a *null* los objetos

Ejemplo.

```
public class ScopeExample {  
    private int i = 1;  
    public void firstMethod() {  
        int i = 4, j = 5;  
        System.out.println(this.i);  
        this.i = i + j;  
        System.out.println(this.i);  
        secondMethod(7);  
        System.out.println(this.i);  
    }  
    public void secondMethod(int i) {  
        this.i = i;  
    }  
}
```

```
public class TestScoping {  
    public static void main (String[] args){  
        ScopeExample sc = new  
        ScopeExample();  
        scope.firstMethod();  
    }  
}
```

- **Salida por pantalla:**

- 1
- 9
- 7

Inicialización de variables.

- Las variables en Java deben ser inicializadas antes de ser utilizadas:
 - La inicialización de las variables de instancia (atributos) se lleva a cabo en el proceso de creación del objeto.
 - Si no se inicializan explícitamente se aplican los valores por defecto establecidos anteriormente
 - Las variables locales deben ser inicializadas manualmente antes de ser utilizadas.
 - El puntero **this** y las variables que son parámetros de métodos se inicializan cuando comienza la ejecución del método en cuestión

```
public void doComputation() {  
    int x = (int) (Math.random() * 100);  
    int y, z;  
    if (x > 50) {  
        y = 9;  
    }  
    z = y + x; //Posible utilización antes de la inicialización.  
}
```

Operadores

- La mayor parte de los operadores en Java son los mismos que los obtenidos desde otros lenguajes
- Operadores aritméticos: * / + - % ++ --
- Operadores relacionales: > >= < <= == !=
- Operadores Lógicos: ! & | ^ → Devuelven un valor “boolean”
- Operadores Lógicos en cortocircuito: || &&
 - Utilidad:
MyDate d;
if ((d!=null) && (d.day <= 31) && (d.month!=null) && (d.month < 13)) {
 //...
}
- Precaución con el uso de los valores lógicos:
int i = 1;
if (i) // genera un error en compilación
if (i==1) // correcto

Operadores bit a bit.

- Son aplicables sobre datos enteros.
- Operadores Lógicos ~ & | ^
- Operadores de desplazamiento a la derecha.

>> Desplazamiento a la derecha o con signo (mantiene el signo. Operaciones aritméticas.

>>> Desplazamiento a la derecha o sin signo (no mantiene el signo. Operaciones lógicas.

128 >> 1 = 64

256 >> 4 = 16

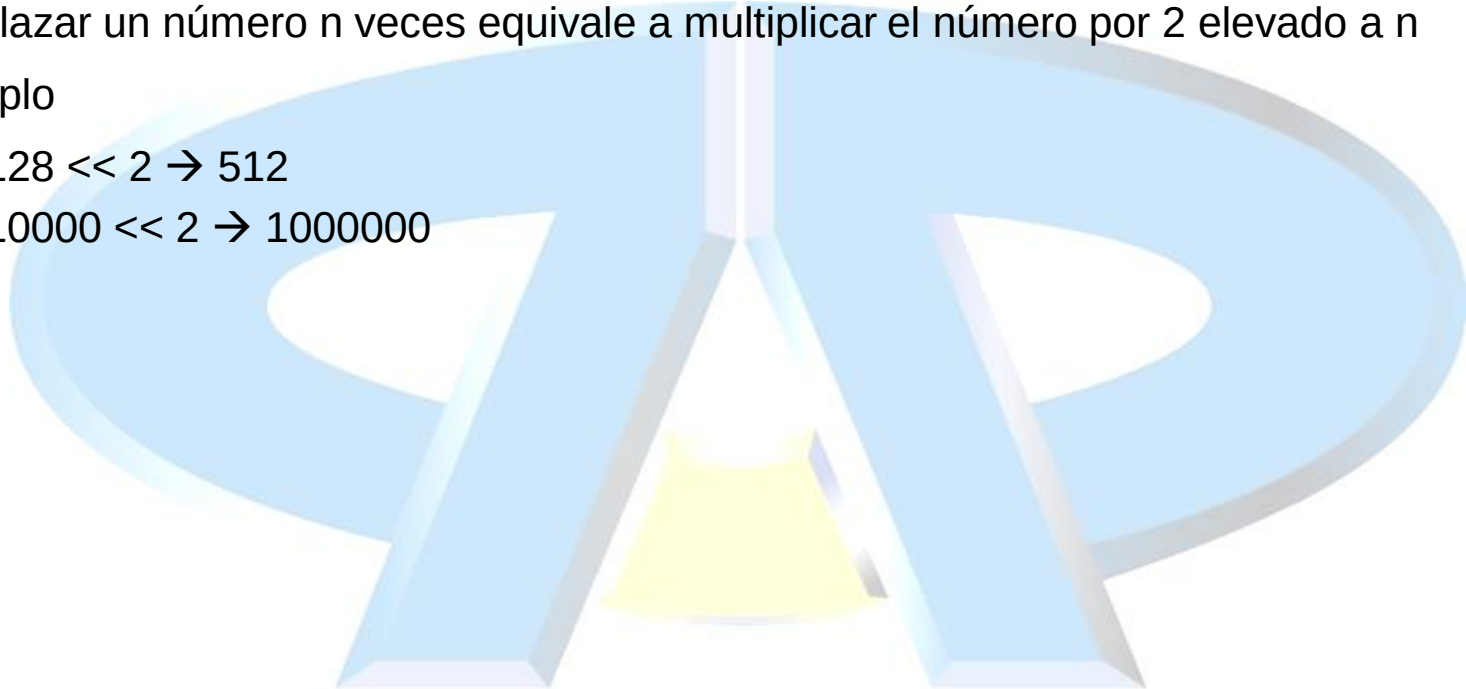
-256 >> 4 = -16

1010 >> 2 = 11/1010

1010 >>> 2 = 00/1010

Operadores bit a bit.

- Operaciones de desplazamiento a la izquierda <<
- Desplazar un número n veces equivale a multiplicar el número por 2 elevado a n
- Ejemplo
 - $128 \ll 2 \rightarrow 512$
 - $10000 \ll 2 \rightarrow 1000000$



Concatenación de cadenas con +

- El operador +:
 - Realiza concatenación de String.
 - Genera un nuevo String:
`String saludo = "Dr";`
`String nombre = "Cándido" + " " + "Pérez";`
`String titulo = saludo + " " + nombre;`
- Solamente es aplicable a String. Si se trata de aplicar a un elemento no String, automáticamente se convierte a String.
- Si se combina con números, el utilizarlo como suma o como concatenación depende de los operandos. Si alguno de los dos es un String, siempre concatena
 - `1 + 2 + "A" + 3 + 4 + "B" → "3A34B"`

Conversiones

- Asignar un valor de un tipo a una variable de otro tipo.
- Si no se pierde información se puede realizar de forma automática.
- Si se pierde información durante la conversión el usuario debe confirmar la conversión.

```
long bigValue = 99L;
```

```
int squashed = bigValue; //Error, se necesita conversión explícita.
```

```
int squashed = (int) bigValue;
```

Promoción y conversión de expresiones.

- Las variables se promueven automáticamente a la forma más grande (int a long).
- Una expresión es compatible para la asignación si el tipo de la variable es al menos tan grande que el tipo de la expresión.

```
long bigVal = 6;      // 6 es un tipo entero.
```

```
int smallVal = 99L;   // 99 es un tipo long. Error.
```

```
double z = 12.414F;   //12.414F es float.
```

```
float z1 = 12.414;     //12.414 es double. Error.
```

Instrucciones condicionales

Sintaxis Básica:

```
if (expresión_booleana) {  
    <sentencia>*  
}
```

```
if (x > 25)  
    System.out.print("X es mayor que 25");
```

Formato recomendado:

```
if (x > 25) {  
    System.out.print("X es mayor que 25");  
}
```

Instrucciones condicionales

Sintaxis Compleja:

```
if (expresión_booleana) {  
    <sentencia>*  
}  
else {  
    <sentencia>*  
}
```

Formato recomendado:

```
if (x > 25) {  
    System.out.print("X es mayor que 25");  
} else {  
    System.out.print("X es menor o igual que 25");  
}
```

Instrucciones condicionales

```
switch (expresion) {  
    case <expresion1>:  
        <sentencias>;  
        break;  
    case <expresion2>:  
        <sentencias>;  
        break;  
    case <expresion3>:  
        <sentencias>;  
        break;  
    default:  
        <sentencias>;  
        break;  
}
```

- La <expresion> debe ser compatible con el tipo “int”, es decir, int o tipos que promocionen a int como byte, short y char. No se admiten ni float ni double ni long.
- Versión 5: Permite utilizar tipos enum
- Versión 7: Permite utilizar String

Instrucciones condicionales

- En la Versión 14, se añadió una nueva versión para las instrucciones SWITCH, permitiendo poner varios valores en cada CASE y que el SWITCH devuelva un valor:

```
boolean isTodayHoliday;  
switch (day) {  
    case "MONDAY":  
    case "TUESDAY":  
    case "WEDNESDAY":  
    case "THURSDAY":  
    case "FRIDAY":  
        isTodayHoliday = false;  
        break;  
    case "SATURDAY":  
    case "SUNDAY":  
        isTodayHoliday = true;  
        break;  
    default:  
        throw new IllegalArgumentException("What's a " + day);  
}
```

```
boolean isTodayHoliday = switch (day) {  
    case "MONDAY", "TUESDAY", "WEDNESDAY", "THURSDAY", "FRIDAY" -> yield false;  
    case "SATURDAY", "SUNDAY" -> yield true;  
    default -> throw new IllegalArgumentException("What's a " + day);  
};
```

Instrucciones para bucles

```
for (expr_init; expr_booleana; expr_alter) {  
    sentencias;
```

```
}  
for (var : colección_elementos) {
```

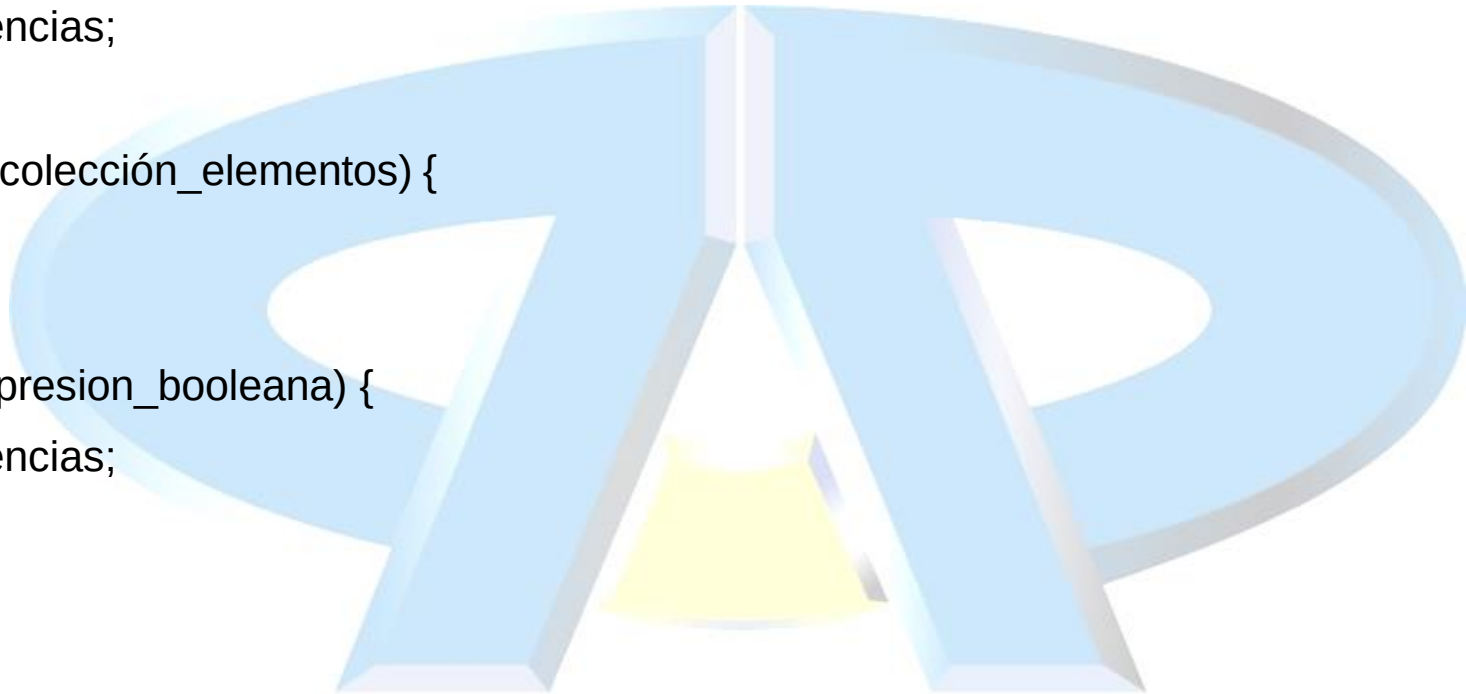
```
}  
//
```

```
while (expresion_booleana) {  
    sentencias;
```

```
}  
//
```

```
do {  
    sentencias;
```

```
} while(expresion_booleana);
```



Control de flujo especial en bucles

`break [label];`

`continue [label];`

`<label>: sentencia;`

